Khmer News Classification Using Traditional Machine Learning Algorithms

Methodology Guide for the Reproducible Companion Notebook (KhNeAC Dataset)

Prepared by: [Author Name]

Year 4 Student, Data Science Program — Course: Fundamentals of Machine Learning

November 10, 2026

Abstract

This guide documents the design of a reproducible Jupyter notebook that implements the complete machine learning workflow for Khmer-language news classification, proposed in the accompanying project proposal. The notebook operationalizes the study on the KhNeAC dataset, which distributes 7,344 labeled Khmer news articles in six preprocessing configurations (raw, clean, normalized, compound-segmented, morpheme-segmented, and KCC-segmented) with fixed train, validation, and test partitions. The notebook is organized into ten documented sections—environment setup, data loading, exploratory analysis, preprocessing, feature extraction, model training, evaluation, insight generation, persistence, and a fork guide—with all design choices centralized in a single configuration cell. Four traditional algorithms (Multinomial Naive Bayes, Logistic Regression, Linear SVM, Random Forest) are compared under Bag of Words and TF-IDF representations at both word and character levels. Hyper-parameters are tuned exclusively on the validation split and the test split is evaluated once, preventing selection leakage. The guide explains what is removed during Khmer-specific preprocessing and why, why character n-grams complement word-level features on unsegmented text, and how the segmentation study turns Khmer word segmentation into a controlled experimental factor.

Keywords: Khmer NLP; reproducibility; text classification; TF-IDF; word segmentation; experimental design

1. Introduction and Objectives

The companion notebook transforms the project proposal into executable research. Its objectives are threefold. First, it implements the six-stage workflow—data collection, preprocessing, feature extraction, model training, evaluation, and interpretation—as a single, runnable artifact. Second, it enforces methodological discipline: fixed official splits, validation-only tuning, one-shot test evaluation, and seeded stochastic components. Third, it is designed for reuse: every design choice (dataset name, text field, feature sizes, n-gram ranges, model grids) is centralized in a configuration cell, so that forking the notebook to another dataset or another set of algorithms requires editing configuration rather than code.

2. Dataset and Loading Strategy

The notebook loads the KhNeAC dataset from the Hugging Face Hub [1, 2]. Each sample provides index_id, title, content, and a six-class label (economic, entertainment, life, politic, sport, technology). Crucially, the same 7,344 articles are released in six preprocessing variants (Table 1), with fixed official partitions of 5,140 training, 661 validation, and 1,543 test articles. Because the underlying articles are identical across configurations, segmentation quality becomes a controlled experimental factor: any performance difference between configurations is attributable to the text-processing pipeline rather than to the data. The main configuration is loaded eagerly, while a cached helper loads the remaining configurations lazily for the segmentation study, so no data is transferred twice.

Table 1. The six KhNeAC configurations and their role in the study.

Configuration	Description	Word boundaries
raw	Original scraped text	No
clean	Whitespace trimmed; invisible Unicode removed	No
normalize	Cleaned + Khmer text normalization	No
com_seg	Normalized + compound-word segmentation	Yes
mor_seg	Normalized + morpheme segmentation	Yes
kcc_seg	Normalized + KCC (character-cluster) segmentation	Yes

3. Preprocessing Design

Khmer web text contains artifacts that add noise dimensions without semantic value. The notebook implements a conservative, language-aware cleaning function in which every decision is documented (Table 2). Two subtleties deserve emphasis. First, Unicode NFC normalization is applied so that visually identical Khmer strings become byte-identical. Second, and critically, Python’s regular-expression word class does not match Khmer

combining marks (vowel signs and the COENG subscript sign U+17D2, Unicode category Mn); a naïve punctuation filter therefore silently destroys Khmer orthography, turning a word such as *phnom penh* into isolated consonants. The notebook's filter explicitly preserves the Khmer mark range U+17B6–U+17D3, and the preprocessing cell includes before/after integrity checks on real article text.

Table 2. Preprocessing decisions implemented in the notebook.

Artifact	Treatment	Rationale
Unicode incompatibility	NFC normalization	Identical strings become identical tokens
URLs, e-mails, HTML tags	Removed	Source metadata, not news content
Zero-width characters (U+200B–U+200D, BOM)	Removed	Invisible; matches dataset's own clean policy
Khmer sentence punctuation	Replaced by space	Acts as a pseudo word boundary
Other symbols	Replaced by space; Khmer marks U+17B6–U+17D3 preserved	Protects vowel signs and COENG subscripts
Arabic and Khmer digits	Mapped to a <num> token	Quantities generalize as a type
Repeated whitespace	Collapsed	Canonical form

Stop-word removal is deliberately omitted: TF-IDF weighting already attenuates high-frequency function words, and premature removal risks deleting short discriminative tokens. The function is idempotent, so it is applied uniformly to all six configurations and keeps the experiment matrix comparable.

4. Feature Extraction Strategy

Documents are represented with term-frequency features: Bag of Words counts raw term occurrences, whereas TF-IDF re-weights them by inverse document frequency, down-weighting terms that occur across many documents and up-weighting category-discriminative ones [4]. The central design point is the interaction between the representation and Khmer segmentation. Word-level features (word n-grams) are meaningful only where segmentation inserted spaces, so they are applied to the three segmented configurations. On unsegmented text a whitespace token is a whole clause, which yields a degenerate vocabulary; character n-grams (2–4 grams with word-boundary padding) are used instead because they learn recurring subword units and are robust to imperfect segmentation. This asymmetry operationalizes hypothesis H5 of the proposal—that character-level features mitigate segmentation errors—and turns the word-versus-character choice into an empirical question rather than an assumption. All vectorizers share a vocabulary cap of 20,000 features, a minimum document frequency of two, and sub-linear TF scaling, with values exposed in the configuration cell for ablation.

5. Models and Experimental Protocol

Four algorithms are compared: Multinomial Naive Bayes as a probabilistic baseline [5, 7], Logistic Regression as a calibrated linear model, Linear SVM as the historically strongest family on sparse text [6], and Random Forest as a non-linear ensemble comparator. Each experiment is a scikit-learn pipeline pairing a vectorizer with a classifier [3], which guarantees that feature fitting never sees evaluation data and renders the whole experiment a single serializable object.

Table 3. Registered experiments in the notebook.

ID	Config	Features	Model
E1	main	BoW	Naive Bayes
E2	main	TF-IDF (word)	Naive Bayes
E3	main	TF-IDF (word)	Logistic Regression
E4	main	TF-IDF (word)	Linear SVM
E5	main	TF-IDF (word)	Random Forest
E6	main	TF-IDF (char)	Linear SVM
E7	main	BoW	Logistic Regression
SEG-*	all six	word / char TF-IDF	Linear SVM

The protocol is leakage-free by construction. A small hyper-parameter grid is tuned on the validation split using macro-averaged F1, the selected configuration is refitted on the training split, and the test split is evaluated exactly once at the end. Macro F1 is the primary metric because it weights the six categories equally; accuracy, weighted F1, per-class reports, and confusion matrices accompany it. The segmentation study retrains the same pipeline across all six configurations, providing a direct test of hypothesis H2.

6. Notebook Architecture and Reproducibility

The notebook contains thirty-one cells organized in ten numbered sections, alternating markdown rationale with code. Stochastic components are seeded (seed 42); optional stratified subsampling (sample_size in the configuration cell) permits a fast smoke test before the full run, which takes approximately ten to twenty minutes on a free CPU runtime, dominated by the Random Forest and the segmentation study. The winning pipeline is serialized with joblib together with a results table, yielding submission artifacts and a deployable classifier. A final fork guide lists the exact configuration edits required to swap datasets, disable title concatenation, change n-gram ranges, or register additional models.

7. Expected Insights

The notebook operationalizes the five pre-registered hypotheses of the proposal: TF-IDF versus BoW (H1), segmentation benefit (H2), Linear SVM versus Naive Bayes (H3), the Politics–Economics confusion pair (H4), and character-level robustness (H5). Each is tested by a specific comparison in the results table or the segmentation study, and the discussion cell provides hedged academic sentence frames for reporting supported, partially supported, or unsupported outcomes. Because values are reported only after execution, the notebook preserves the integrity principle of the proposal: no performance figure is asserted before it is measured.

8. Conclusion

The companion notebook provides a rigorous, reusable implementation of the proposed study. Its contribution is less the individual classifiers than the experimental discipline: controlled use of six segmentation configurations, Khmer-aware preprocessing with verified orthographic integrity, representation choices justified by segmentation status, and a leakage-free evaluation protocol. The artifact is intended to be forked—to other Khmer corpora, to additional algorithms, or to a comparison against pretrained language models—while retaining the dataset citation and the reproducibility guarantees.

References

[1] S. Heang (sopagnaheang), "KhNeAC: Khmer News Article Classification Dataset," Hugging Face. [Online]. Available: https://huggingface.co/datasets/sopagnaheang/KhNeAC (accessed Nov. 10, 2026).

[2] CADT IDRI, "Khmer News Classification (unsegmented)," Hugging Face. [Online]. Available: https://huggingface.co/datasets/CADT-IDRI/Khmer_News_classification (accessed Nov. 10, 2026).

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.

[5] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in *Proc. AAAI-98 Workshop on Learning for Text Categorization*, 1998, pp. 41–48.

[6] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *Proc. ECML-98*, 1998, pp. 137–142.

[7] H. Zhang, "The optimality of Naive Bayes," in *Proc. FLAIRS-17*, 2004.

[8] "Pretrained models and evaluation data for the Khmer language," in *Proc. COLING 2022*, Gyeongju, Republic of Korea, 2022.